

# Seguridad - Horizon Medical WebSocket Server (hrzmed\_wss)

**Fase 2 del Plan de Mejoras** — Seguridad Básica Backend  
Implementada sobre la base corregida de la Fase 1 (fix aVF).

## Resumen de Cambios

| Área                        | Estado Anterior           | Estado Actual                                   |
|-----------------------------|---------------------------|-------------------------------------------------|
| Autenticación WebSocket     | ✗ Sin autenticación       | ✓ JWT con múltiples métodos de entrega          |
| Tokens/Secretos             | ⚠ Algunos hardcodeados    | ✓ Todo en variables de entorno (.env)           |
| Validación de datos médicos | ✗ Sin validación          | ✓ Validación completa de señales EKG            |
| Rate limiting               | ✗ Sin límites             | ✓ Por IP y por conexión                         |
| Logging de seguridad        | ✗ Solo console.log básico | ✓ Winston estructurado con eventos de seguridad |
| Manejo de errores           | ✗ Mínimo                  | ✓ Graceful shutdown, errores tipados            |

## 1. Autenticación JWT

### Descripción

Todas las conexiones WebSocket requieren un token JWT válido para establecer la conexión. La autenticación ocurre durante el HTTP upgrade handshake, **antes** de que se complete la conexión WebSocket.

## Métodos de Entrega del Token

| Método                 | Formato                                        | Caso de Uso                                |
|------------------------|------------------------------------------------|--------------------------------------------|
| Query Parameter        | <code>ws://host?token=&lt;JWT&gt;</code>       | Dispositivos IoT, clientes simples         |
| Authorization Header   | <code>Authorization: Bearer &lt;JWT&gt;</code> | Aplicaciones web, APIs                     |
| Sec-WebSocket-Protocol | <code>jwt, &lt;token&gt;</code>                | Navegadores (limitación de headers custom) |

## Claims Requeridos del Token

```
{
  "sub": "device-001",      // o "userId" o "deviceId" – al menos uno requerido
  "role": "device",        // "device" | "monitor" | "admin"
  "deviceId": "holter-001", // Opcional: ID del dispositivo Holter
  "patientId": "patient-001", // Opcional: ID del paciente
  "permissions": ["send_ecg", "send_status"],
  "iat": 1234567890,       // Automático
  "exp": 1234654290       // Automático según JWT_MAX_AGE
}
```

## Roles y Permisos

| Rol     | Permisos                                    | Descripción                            |
|---------|---------------------------------------------|----------------------------------------|
| device  | send_ecg , send_status                      | Dispositivos Holter IoT                |
| monitor | monitor                                     | Dashboards de monitoreo en tiempo real |
| admin   | send_ecg , send_status ,<br>monitor , admin | Administradores del sistema            |

## Generar Tokens

```
# Generar token para dispositivo
JWT_SECRET=<secret> node scripts/generate-token.js --role device --deviceId holter-001

# Generar token para monitor
JWT_SECRET=<secret> node scripts/generate-token.js --role monitor --userId monitor-001

# Token con expiración personalizada
JWT_SECRET=<secret> node scripts/generate-token.js --role admin --userId admin-001 --
expires 8h
```

## Desactivar Autenticación (Solo Desarrollo)

```
AUTH_ENABLED=false npm run dev
```

⚠ **NUNCA desactivar en producción.**

## 2. Variables de Entorno

Toda configuración sensible se maneja vía variables de entorno. Ver `.env.example` para la plantilla completa.

### Variables Críticas de Seguridad

| Variable                                | Descripción                                  | Requerida |
|-----------------------------------------|----------------------------------------------|-----------|
| <code>JWT_SECRET</code>                 | Clave secreta para firmar/verificar JWT      | ✓ Sí      |
| <code>MONGO_INITDB_ROOT_PASSWORD</code> | Contraseña root de MongoDB                   | ✓ Sí      |
| <code>MONGO_PASSWORD</code>             | Contraseña del usuario de aplicación MongoDB | ✓ Sí      |

### Generar JWT\_SECRET Seguro

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

### Archivos Protegidos

- `.env` — Excluido de git via `.gitignore`
- `.env.example` — Plantilla sin valores reales (sí en git)

## 3. Validación de Datos Médicos (EKG)

### Validaciones Implementadas

#### Nivel de Mensaje

- Tamaño máximo: 64KB (prevención DoS)
- Formato JSON válido requerido
- Campo `type` obligatorio y validado contra lista blanca

#### Datos ECG ( `type: "ecg_data"` )

- **deviceId**: String no vacío, máximo 128 caracteres
- **channelId**: Debe ser uno de `8171 - 8178` (UUIDs BLE del ADS1298)
- **samples**: Array de enteros dentro del rango ADC de 24 bits:
  - Mínimo: `-8388608` ( $-2^{23}$ )
  - Máximo: `8388607` ( $2^{23} - 1$ )
- Mínimo 1 muestra, máximo 100 por paquete
- **timestamp**: Unix ms o ISO 8601, con advertencia si drift > 5 minutos
- **sequenceNumber**: Entero no negativo (opcional)

**Estado de Dispositivo ( type: "device\_status" )**

- Status válidos: `online` , `offline` , `low_battery` , `error` , `calibrating`
- `batteryLevel` : 0-100

**Info de Paciente ( type: "patient\_info" )**

- **Bloqueo de PII**: Los campos `ssn` , `socialSecurity` , `creditCard` , `bankAccount` son **rechazados** con evento de seguridad
- Nombres truncados a 100 caracteres

---

## 4. Rate Limiting

| Parámetro                   | Default | Variable de Entorno                   |
|-----------------------------|---------|---------------------------------------|
| Conexiones por IP           | 10      | <code>MAX_CONNECTIONS_PER_IP</code>   |
| Mensajes por segundo        | 50      | <code>MAX_MESSAGES_PER_SECOND</code>  |
| Fallos de auth antes de ban | 5       | <code>MAX_AUTH_FAILURES_PER_IP</code> |
| Duración del ban            | 15 min  | <code>BAN_DURATION_MS</code>          |

---

## 5. Logging de Seguridad

### Eventos de Seguridad Registrados

| Evento                         | Nivel | Descripción                     |
|--------------------------------|-------|---------------------------------|
| AUTH_NO_TOKEN                  | WARN  | Conexión sin token              |
| AUTH_FAILED                    | WARN  | Token inválido/expirado         |
| AUTH_MISSING_CLAIMS            | WARN  | Token sin claims de identidad   |
| AUTH_CONFIG_ERROR              | WARN  | JWT_SECRET no configurado       |
| CONNECTION_REJECTED_BANNED_IP  | WARN  | IP baneada por fallos repetidos |
| CONNECTION_REJECTED_RATE_LIMIT | WARN  | Límite de conexiones por IP     |
| MESSAGE_RATE_LIMITED           | WARN  | Límite de mensajes por segundo  |
| PII_TRANSMISSION_ATTEMPT       | WARN  | Intento de enviar datos PII     |
| IP_BANNED                      | WARN  | IP baneada automáticamente      |
| INVALID_DATA_RECEIVED          | INFO  | Datos médicos inválidos         |

### Formato del Log

```
2026-05-15 10:30:45.123 [WARN] {"service":"hrzmed-wss","security":true,"event":"AUTH_FAILED","ip":"192.168.1.100","reason":"Token expired"} [SECURITY] AUTH_FAILED
```

En producción, los logs se escriben en:

- /var/log/hrzmed/error.log — Solo errores
- /var/log/hrzmed/security.log — Eventos de seguridad (nivel warn+)

## 6. Health Check

Endpoint HTTP para monitoreo:

```
GET /health
```

Respuesta:

```
{
  "status": "healthy",
  "mongo": "connected",
  "uptime": 3600,
  "connections": 5,
  "timestamp": "2026-05-15T10:30:00.000Z"
}
```

## 7. Estructura de Archivos (Post-Fase 2)

```
hrzmed_wss/
├── index.js                                # Servidor principal (refactorizado)
├── src/
│   ├── middleware/
│   │   ├── auth.js                        # Autenticación JWT
│   │   └── rateLimiter.js                 # Rate limiting por IP/conexión
│   ├── validators/
│   │   └── ecgValidator.js                # Validación de datos médicos
│   └── utils/
│       └── logger.js                      # Logging estructurado (Winston)
├── tests/
│   ├── auth.test.js                      # Tests de autenticación (12 tests)
│   └── ecgValidator.test.js              # Tests de validación (22 tests)
├── scripts/
│   └── generate-token.js                  # Utilidad para generar JWT tokens
├── .env.example                           # Plantilla de variables de entorno
├── .gitignore                             # Excluye .env y node_modules
├── docker-compose.yml                     # Actualizado con nuevas env vars
├── Dockerfile                             # Sin cambios
├── client.js                              # Cliente ejemplo con JWT auth
├── SECURITY.md                             # Este documento
└── README.md                             # Documentación general
```

## 8. Mejoras Futuras (Fase 3+)

- [ ] Almacenamiento de datos ECG validados en MongoDB (Mongoose schemas)
- [ ] Refresh tokens y rotación automática
- [ ] Encriptación end-to-end de datos de pacientes
- [ ] Auditoría completa de acceso a datos (HIPAA compliance)
- [ ] Rate limiting distribuido (Redis) para múltiples instancias
- [ ] Certificados mTLS para dispositivos IoT
- [ ] Integración con sistema de identidad centralizado (OAuth2/OIDC)